LLM ENGINEERING

Week 05

Optimization

Bethgelab · Thursday, 12 November 2026

University of Tübingen · Winter term 2026/27

Tübingen AI Center



Announcements

Quiz 2 on ILIAS

- Weeks 3, 4 and Bonus 2
- Due Friday 13 November, end of day

Fourth exercise: this lecture

- Due Wednesday 18 November, 22:00
- Email when it reaches your repository

The plan until February changes

- Cluster down 23 November to 4 December
- New Kids comes as a video, 3 December
- No lecture on 10 December, tutorial on Zoom
- Free week for your project: ideas, pitch, first experiments
- And a fun Christmas surprise event, more on both next week



New dates until February

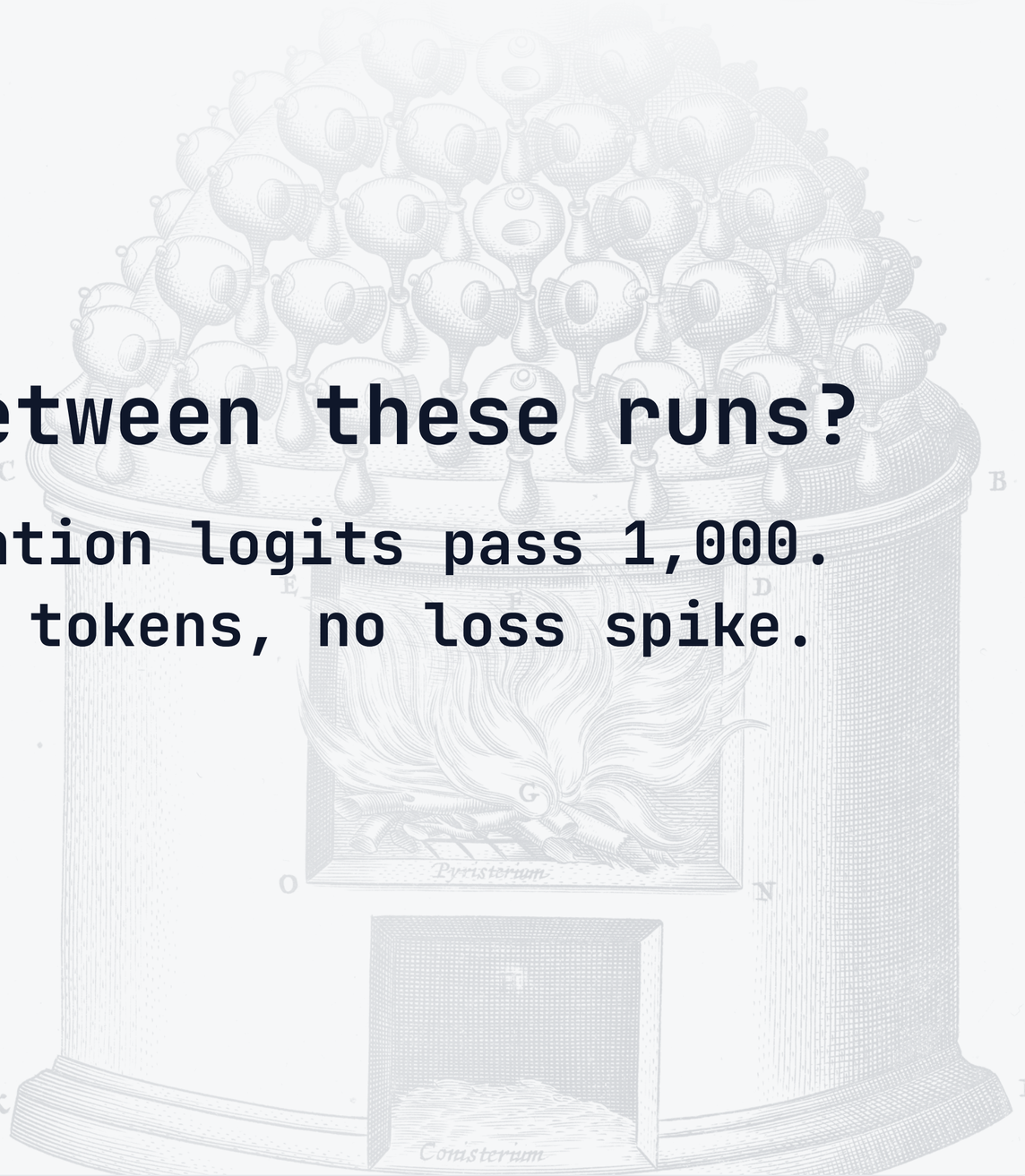
Date	Lecture	Exercise due
19 Nov	Week 6: Parallelization	25 Nov
26 Nov	Week 7: Scaling laws	16 Dec
3 Dec	Week 8: New Kids (video)	none
10 Dec	No lecture: NeurIPS	none
17 Dec	Week 9: Data curation	6 Jan
7 Jan	Week 10: Inference	13 Jan
14 Jan	Week 11: Post-training I	20 Jan
21 Jan	Week 12: Post-training II	27 Jan
28 Jan	Week 13: Mixture of experts	3 Feb
4 Feb	Week 14: Guest lecture (planned)	none

- Pass any 6 of 11 exercises
- Quiz 6 moves to Friday 5 February

What changed between these runs?

Test run, 53B: attention logits pass 1,000.

Main run, 1T: 15.5T tokens, no loss spike.

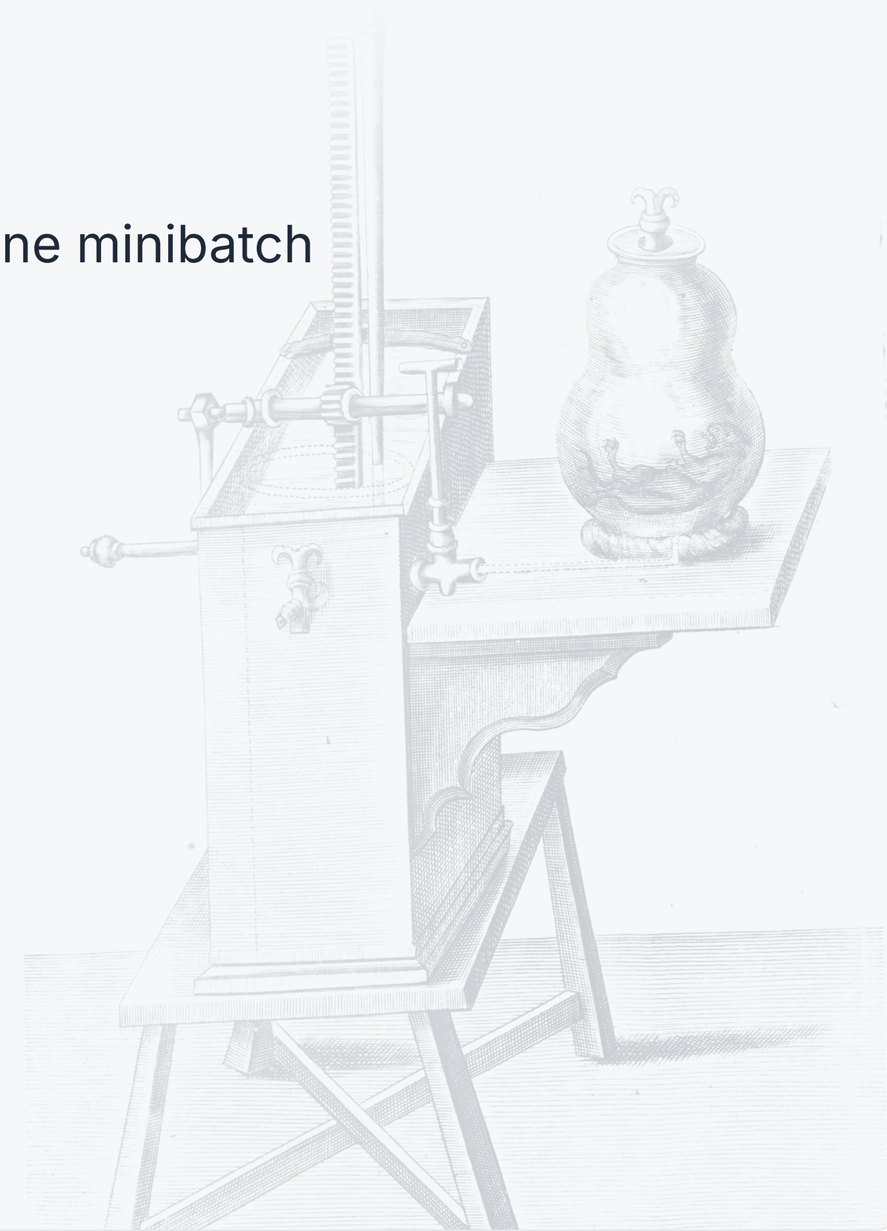


What keeps a long training run stable?

- Optimizer: how each update is shaped
- Schedule: learning rate and batch over time
- Monitoring: signals that warn early
- Controls: bound extremes where they form
- Precision: range and accuracy per tensor
- This week: build all five into your trainer

Training steps against the loss gradient

- Goal: lower the loss over the parameters
- Step against the gradient: $\theta \leftarrow \theta - \eta \cdot g$
- Stochastic gradient descent (SGD): gradient of one minibatch
- One learning rate for every coordinate



Adam adapts each coordinate's step

(1)
$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\, g_t \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2)\, g_t^2$$

(2)
$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \qquad \theta_t = \theta_{t-1} - \eta\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

- m: exponential moving average (EMA) of gradients, averages out noise
- v: EMA of squared gradients, sets each coordinate's step
- Bias correction: both start at zero, t counts steps
- Adam: a diagonal preconditioner

AdamW decouples weight decay from the adaptive step

(3)
$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \, \theta_{t-1} \right)$$

- An L2 penalty in the gradient gets divided by $\sqrt{\hat{v}_t}$
- Same λ, but each coordinate shrinks by a different amount
- AdamW, formula (3): decay outside the adaptive step
- Decay ηλ falls with the learning rate (LR)

Muon starts from matrix-shaped updates

- Hidden weights: 2D matrices, rows and columns
- Flattening throws that structure away
- **Muon** treats momentum as a matrix
- Keller Jordan's blog, December 2024 (Jordan et al., 2024)

How uneven is real momentum?

Muon, our Week 5 model, after 4,000 steps:
the momentum of one 1,536 × 512 MLP matrix.
Largest singular value ÷ median one:
about 2, 5, 30 or 300?

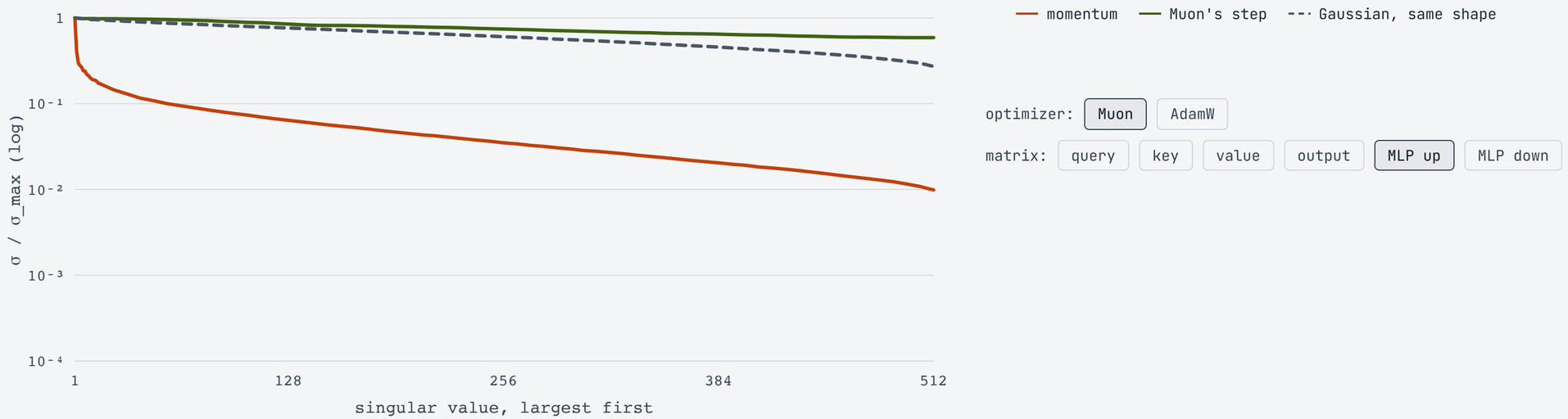
Momentum concentrates in a few directions

Measured: Week 5 model, block 3, step 4,000

MLP up matrix, 1,536 × 512

largest σ ÷ median σ

momentum	**29×**
Muon's step	**1.3×**
Gaussian	1.7×



optimizer: Muon | AdamW

matrix: query | key | value | output | MLP up | MLP down

- About 30: a few directions dominate
- Gaussian noise of the same shape: 1.7
- Muon's step: 1.3, explained next

One global step size starves the quiet directions

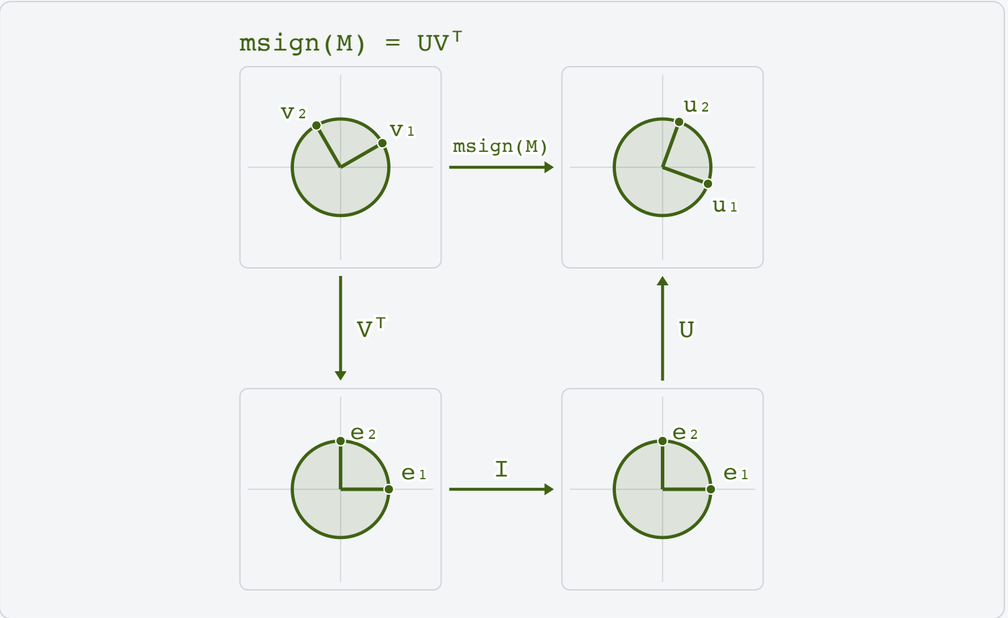
- Momentum and Adam steps: a few loud directions (Jordan et al., 2024)
- One scale: loud directions dominate
- Wanted: an equal step per direction

Muon equalizes the update's singular directions

(4) $$M = U\Sigma V^\top \qquad \operatorname{msign}(M) = UV^\top$$

(4a) $$M_t = \beta M_{t-1} + G_t \qquad W_t = W_{t-1} - \eta \operatorname{msign}(M_t)$$

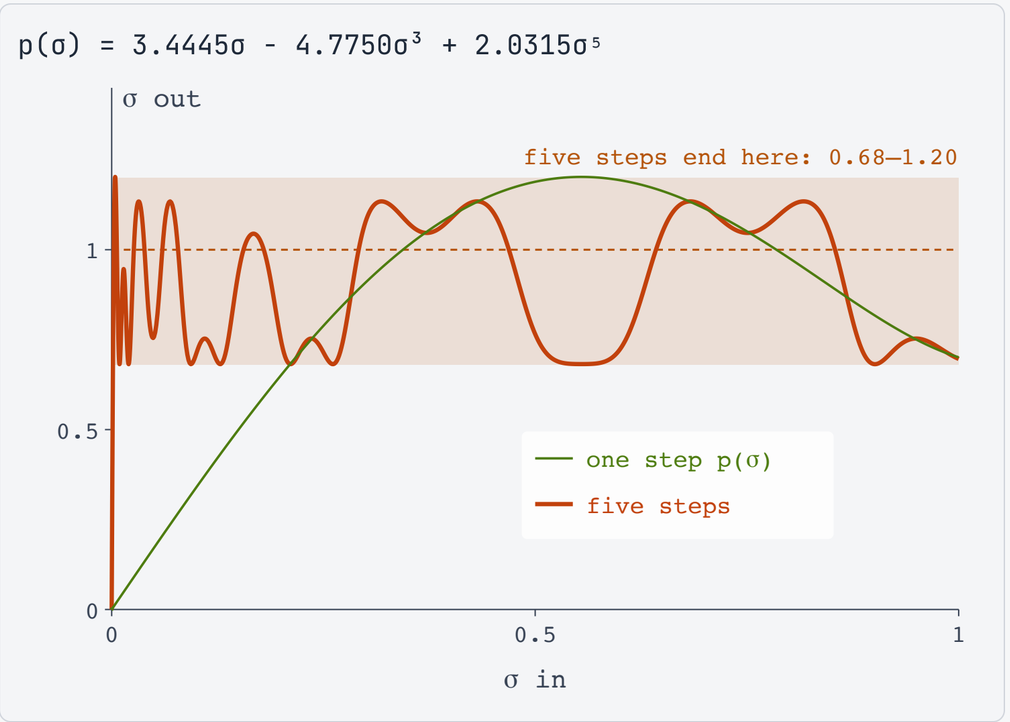
- Singular value decomposition (SVD): rotations and gains
- msign keeps U and V, sets every σ to 1
- Orthogonalization: an equal step per direction
- Muon never computes the SVD itself



Newton-Schulz makes the matrix sign practical

$$X_{k+1} = a\,X_k + b\,(X_k X_k^\top)\,X_k + c\,(X_k X_k^\top)^2\,X_k \tag{5}$$

- A full SVD every step: too expensive
- Newton–Schulz: matrix multiplications only, fast on GPUs
- Each step maps every σ to p(σ)
- Five steps: every σ above 0.002 ends near 1



Muon targets matrices; AdamW handles the rest

```
def newton_schulz5(G, steps=5):            # approximate msign, matmuls only
    a, b, c = (3.4445, -4.7750, 2.0315)
    X = G.bfloat16() / (G.norm() + 1e-7)   # into [0, 1]; the course stays fp32
    for _ in range(steps):
        A = X @ X.T
        X = a * X + (b * A + c * A @ A) @ X
    return X

def muon_update(grad, buf, beta=0.95):     # one hidden weight matrix
    buf.mul_(beta).add_(grad)              # momentum buffer
    g = grad + beta * buf                  # Nesterov-style blend
    return newton_schulz5(g.reshape(len(g), -1)).view_as(g)

W -= lr * muon_update(W.grad, buf)         # LR-scaled step
```

$$0.2\sqrt{\max(\text{fan_in}, \text{fan_out})}$$

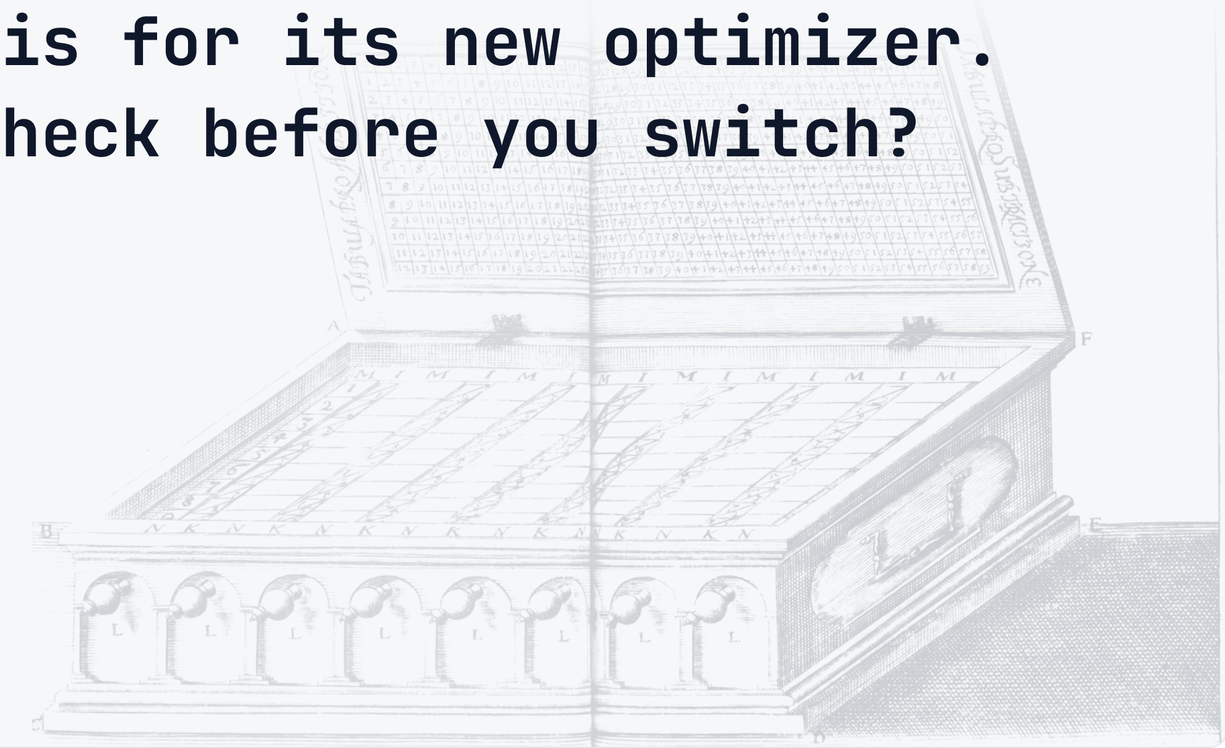
- Hidden 2D matrices: Muon
- Embeddings, head, norms, scalars: AdamW
- One momentum buffer, no v: half of AdamW's state
- Update rescaled to AdamW's root mean square (RMS) (Liu et al., 2025)

Four practices keep Muon stable at scale

- Scale each update by its matrix shape
- Keep weight decay: weights grow otherwise
- Treat each head and each expert as its own matrix
- Watch the max logit, and bound it

Half of AdamW's compute?

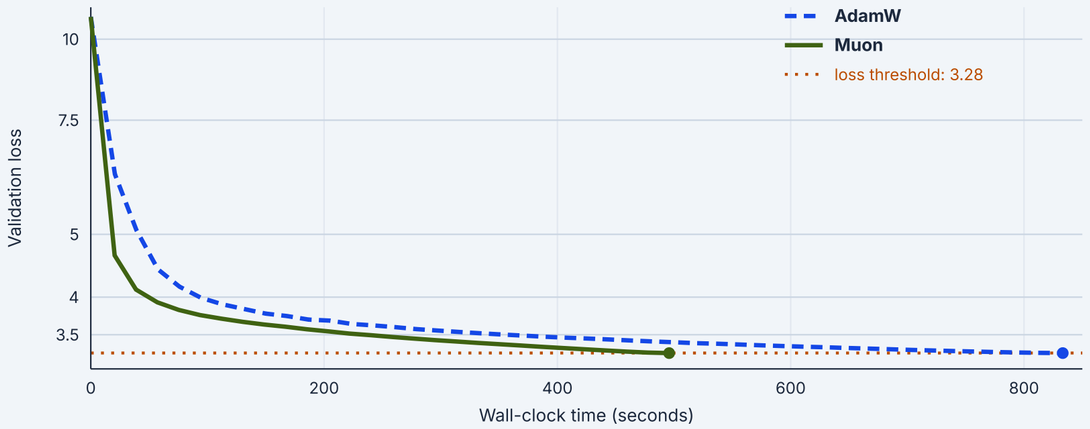
A paper reports this for its new optimizer.
What would you check before you switch?



How strong is the evidence for Muon?

- Speedrun: GPT-2 small on FineWeb, fewest steps to loss 3.28
- Muon: mean of 20 runs, AdamW: one undertuned run
- Moonlight: matched AdamW with about half its compute
- Wen et al.: speedup 1.4× at 0.1B, 1.1× at 1.2B

Keller Jordan's modded-nanoGPT optimizer speedrun: validation loss versus wall-clock time



Benchmark rule
loss < 3.28 and (3.28 − mean)√n ≥ 0.004 · score: training steps

Muon
3,375 steps · 3.27893 mean loss · 495.817 s mean · n=20 · clears the record rule

AdamW
5,625 steps · 3.27903 loss · 833.211 s · n=1 · reaches loss below 3.28 but fails the significance test

K2 traced its logit instability to Muon

- 53B test run: logits pass 1,000
- Saturated softmax weakens attention gradients
- K2: logit explosion more frequent with Muon than AdamW (Kimi Team, 2025)
- K2 skips qk-norm (normalizes q, k): MLA never forms keys (Kimi Team, 2025)

QK-Clip rescales only the offending head

(6)

$$\gamma_h = \min\left(1,\ \frac{\tau}{S^h_{\max}}\right) \qquad W_q^h \leftarrow \sqrt{\gamma_h}\, W_q^h, \quad W_k^h \leftarrow \sqrt{\gamma_h}\, W_k^h$$

- Measured every step: each head's max logit S
- Chosen: threshold τ = 100; above it, scale W_q, W_k by √γ
- K2 run: 12.7% of heads clipped, then none (Kimi Team, 2025)
- MuonClip: Muon, weight decay, RMS matching, QK-Clip (Kimi Team, 2025)

Warmup protects the unstable first steps

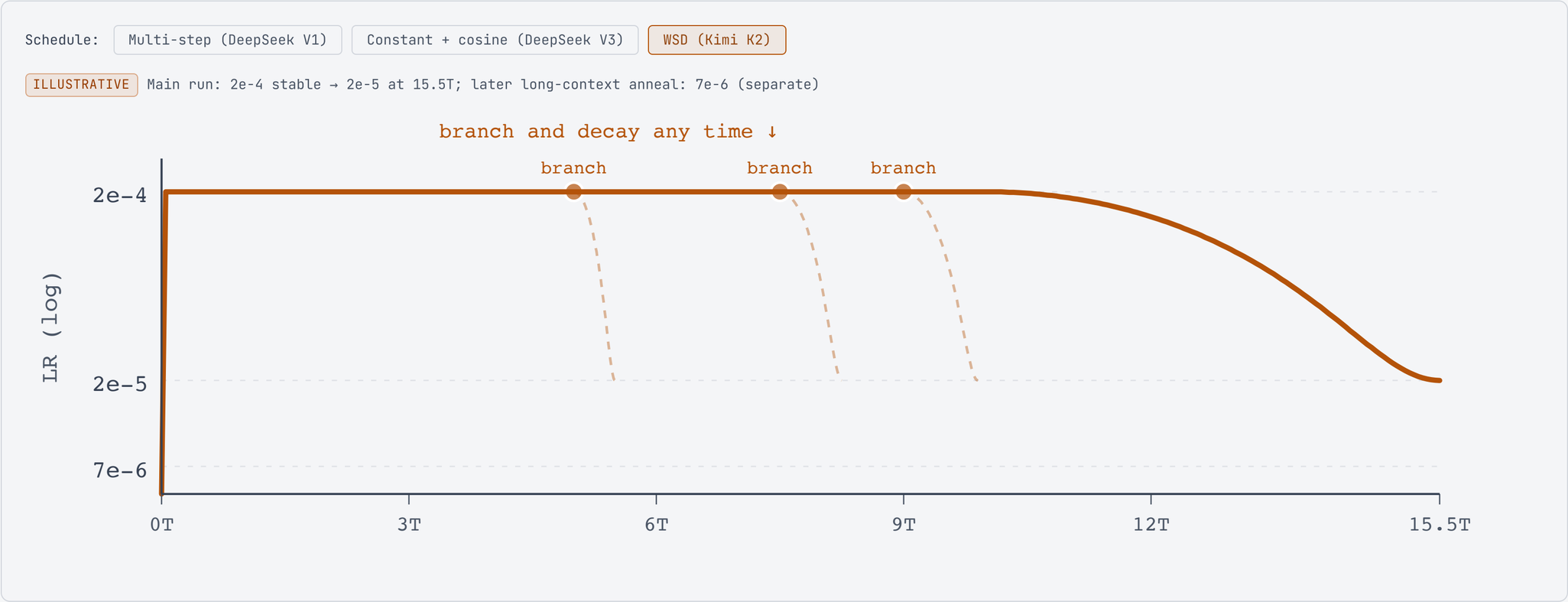
- Early $\hat{v}$: built from a few gradients, so noisy
- Near initialization: high curvature, so full steps overshoot
- Warmup: raise the LR linearly from near zero
- K2: 500 steps, DeepSeek V3: 2,000 steps

WSD separates training from checkpoint decay

- Cosine: the decay is tied to a fixed run length
- Warmup-stable-decay (WSD): named by MiniCPM (Hu et al., 2024)
- Stable: constant LR, every checkpoint can start a decay
- Decay: branch a leg when you need one
- K2: 2e-4 for 10T tokens, then decay over 5.5T (Kimi Team, 2025)

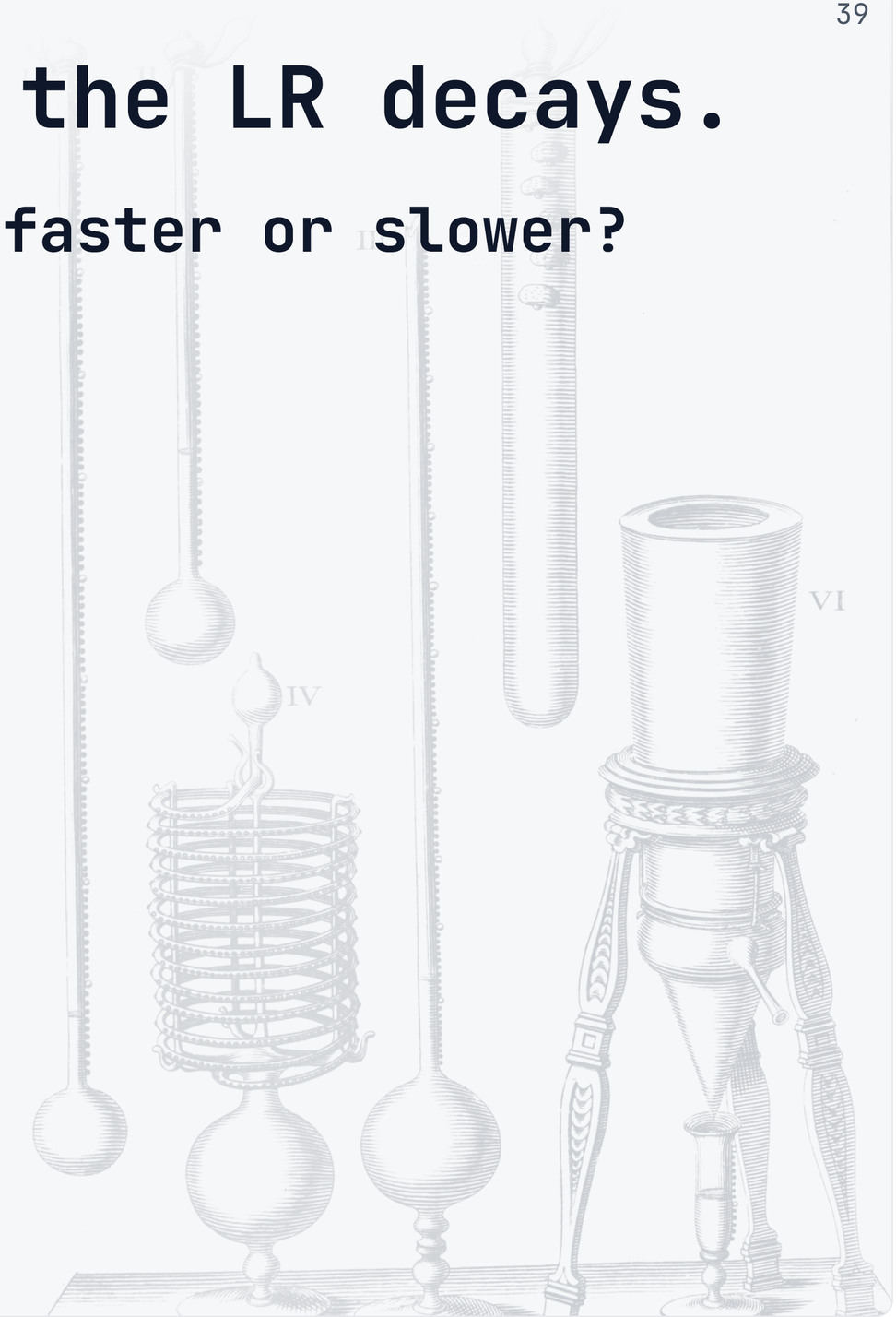
Branch a decay leg without ending the stable run

- Solid: K2's reported 15.5T run
- Dashed: illustrative decay legs, each 10% of its stable span (Hu et al., 2024)
- Open question: does skipping decay help fine-tuning?
- Yano et al.: one lab, 1B and 8B, SFT only (Yano et al., 2026)



At 70% of the run, the LR decays.

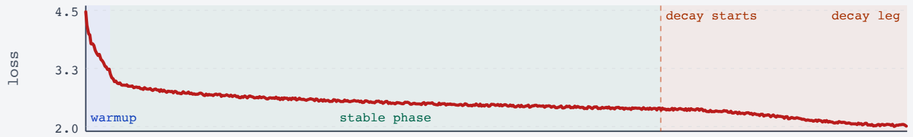
Does the loss now fall faster or slower?



Decaying the learning rate drops the loss

ILLUSTRATIVE Healthy WSD training run: dashboard traces

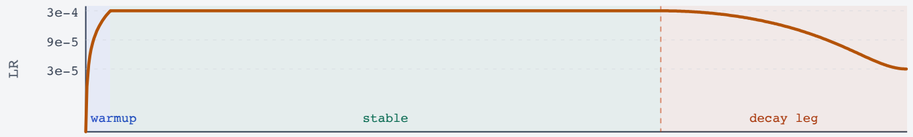
Loss (cross-entropy)



Pre-clip gradient norm



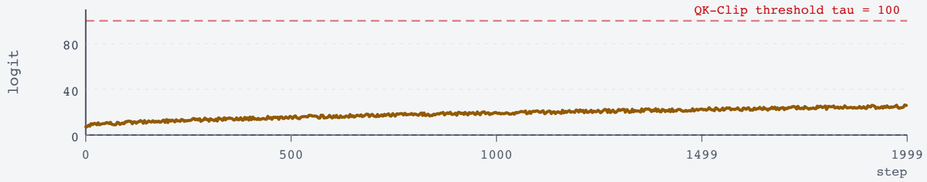
Learning rate (log scale)



Throughput (tokens/s)



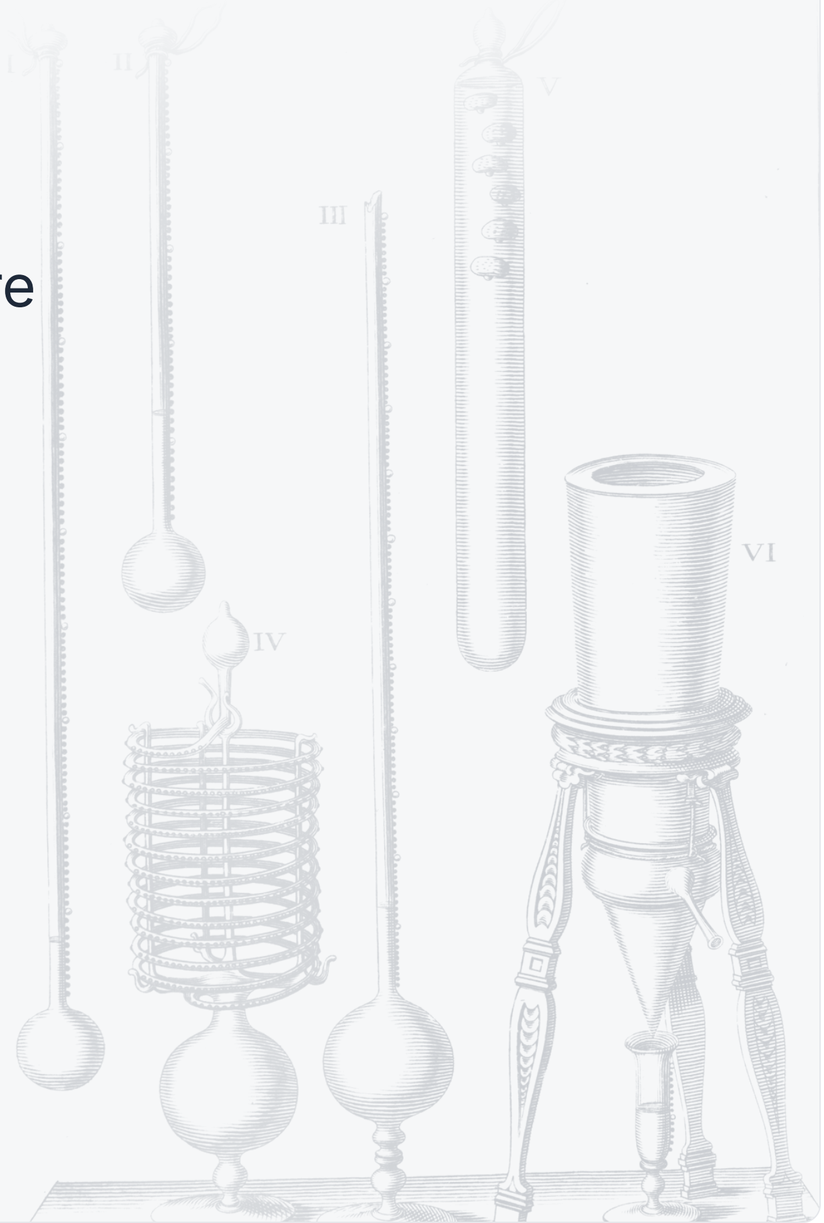
Max attention logit



- Faster: the loss drops 0.35 against 0.12 over equal spans
- Phases: fast drop, slow middle, decay drop
- Loss across tokenizers: not comparable

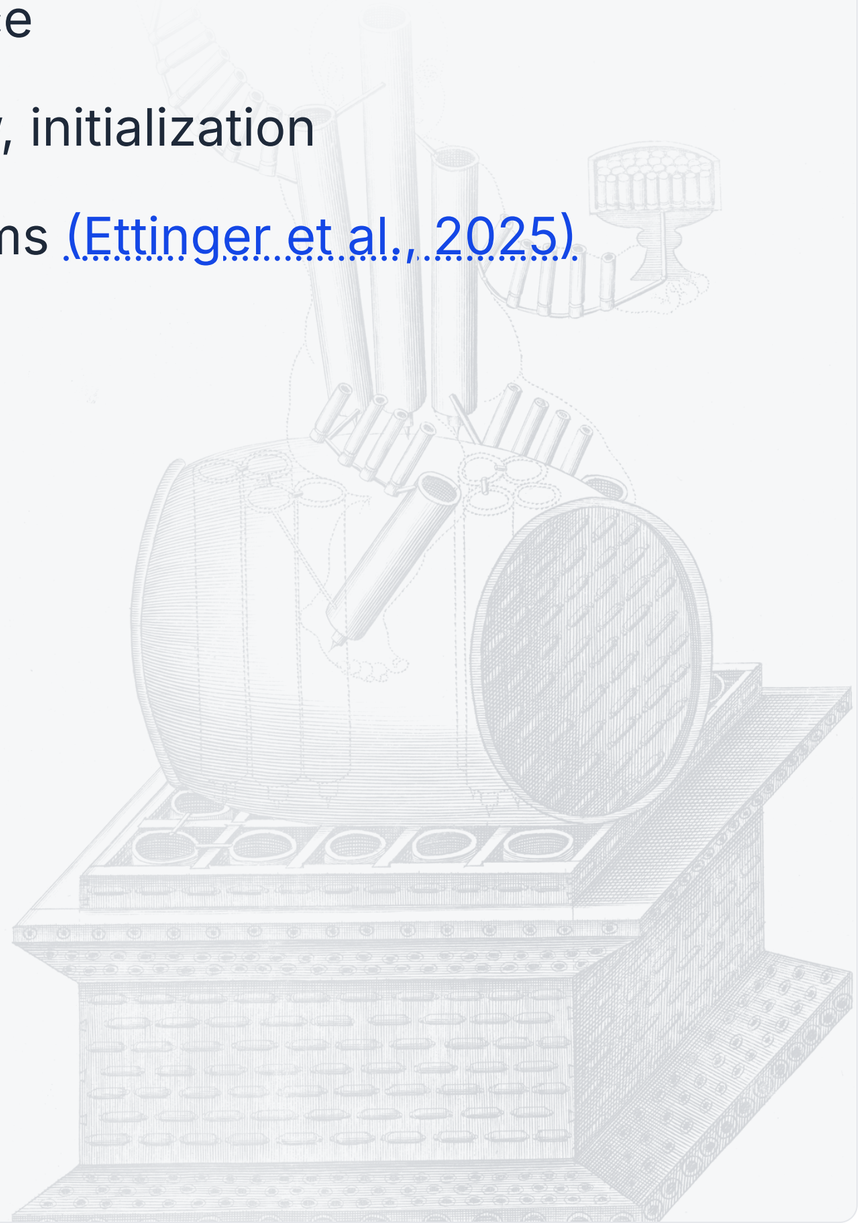
Which signal leads depends on the failure

- Log the gradient norm before clipping
- A healthy norm drifts slowly within a band
- Which signal leads differs from failure to failure



Loss spikes have several trajectories and causes

- Shapes: fast recovery, slow recovery or divergence
- Causes: bad batches, logit growth, fp16 underflow, initialization
- OLMo drops documents with 32+ repeated n-grams (Ettinger et al., 2025)
- The trace alone does not name a cause



The spike score makes detection operational

(8)
$$\text{spike score} = \frac{1}{T-128} \sum_{t=129}^{T} \mathbf{1}\left[\, n_t \geq \mu(n_{t-128:t-1}) + 6\,\sigma_{\text{pop}}(n_{t-128:t-1}) \,\right]$$

- Preceding 128 norms: population mean and standard deviation (Merrill et al., 2026)
- Flag: at least 6σ above the mean
- Score: share of eligible steps flagged
- Course trainer: a z-score of each step's loss

Rollback, skip, and resume after a spike

- Roll back to a clean checkpoint
- Skip batches (PaLM) or change the recipe
- DeepSeek V4: rollback alone, spikes came back (DeepSeek-AI, 2026a)



Match each control to its intervention point

- QK-Clip: projection weights, after the optimizer step
- qk-norm: Q and K, before the dot product
- Softcapping: squash the logits below a cap c

$$\text{logits} \leftarrow c \cdot \tanh\left(\frac{\text{logits}}{c}\right) \tag{9}$$

- z-loss: output logits, through the loss

$$\mathcal{L}_z = \lambda_z \, \log^2 Z, \qquad Z = \sum_i e^{z_i} \tag{10}$$

- SwiGLU clamp: MLP activations
- Gradient clipping: the global gradient norm

A bf16 weight equals 1.0.

The optimizer adds 0.001.
What does bf16 store?



Small updates vanish in a bf16 weight

$$\text{spacing at } 1.0 = 2^{-7} \approx 0.0078$$

- bf16 stores 1.0: the update rounds away
- Any update below half a spacing is lost
- Fix: an fp32 master copy, compute stays bf16

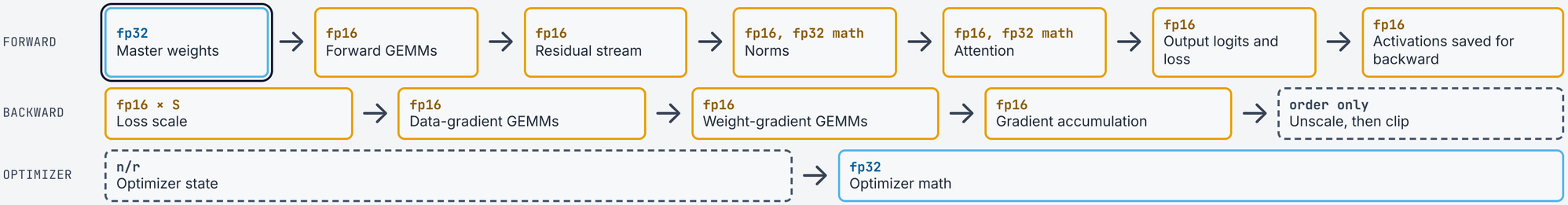
Mixed precision keeps selected operations in fp32

- Why not fp32: bf16 matmuls ~15× faster on an H100
- Matrix multiplications: bf16
- Master weights, optimizer state, gradient accumulation: fp32

Per tensor: what range, what precision?

Where each number format is used in one training step

`fp16 + loss scaling (2018)` | `FP8: DeepSeek V3 / V4` | `FP8: MAI-Thinking-1` | `bf16: course trainer`



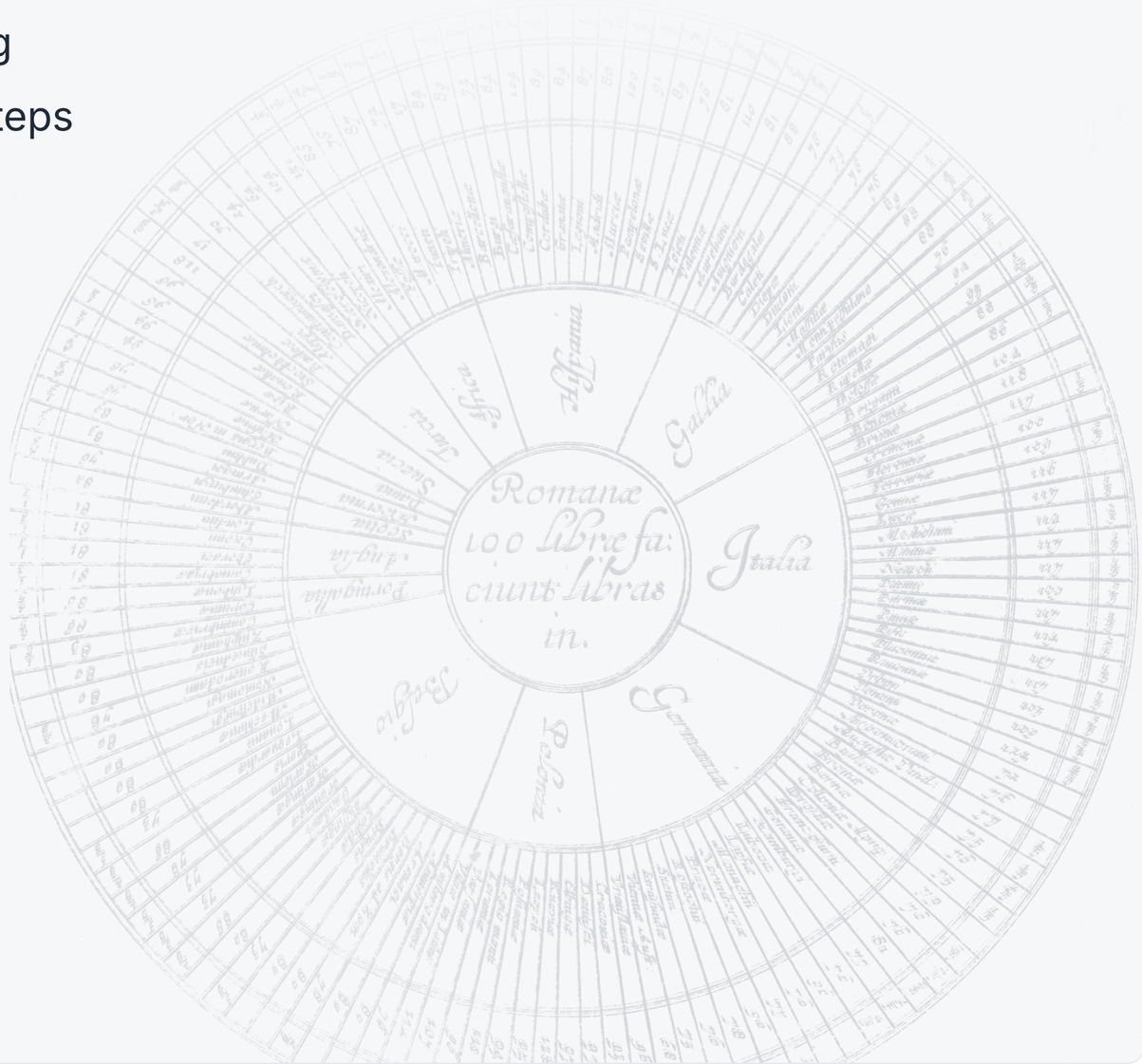
Master weights, fp32. An update smaller than 2^{-24} becomes zero in fp16, and an update 2,048 times smaller than its weight is shifted out when the two are added. (§3.1)

Source: Micikevicius et al., 2018

fp32 bf16 fp16 FP8 E4M3 FP8 E5M2 not reported or not used

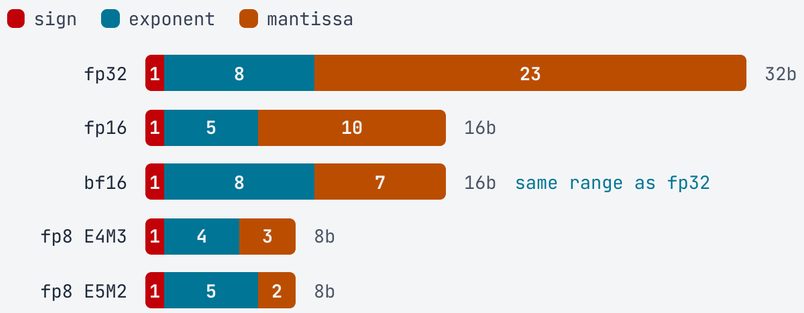
bf16 usually avoids fp16's range-driven loss scaling

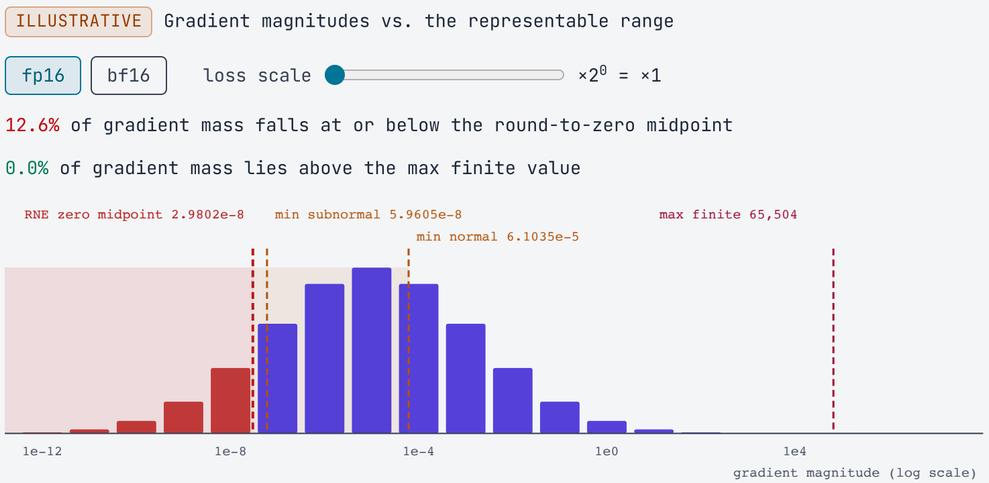
- Same 16 bits: fp16 favors precision, bf16 range
- fp16: tiny gradients round to zero
- Scale the loss, then unscale before clipping
- Dynamic scaler: double after 2,000 finite steps
- Overflow: skip the step, halve the scale



Watch loss scaling move gradients into range

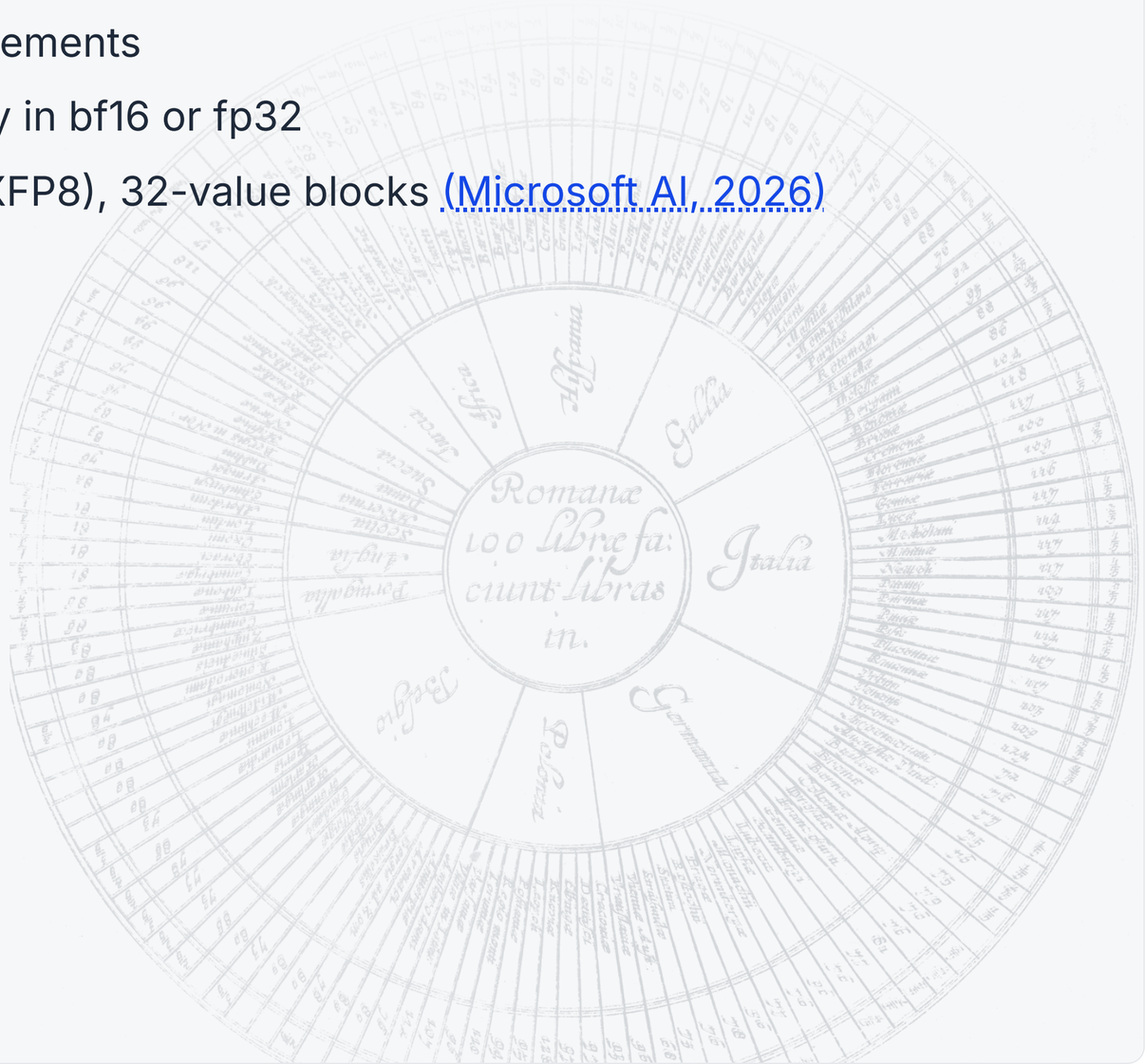
- Raise the scale: less mass underflows
- bf16 moves the underflow threshold far off the axis





FP8 needs local scaling and high-precision islands

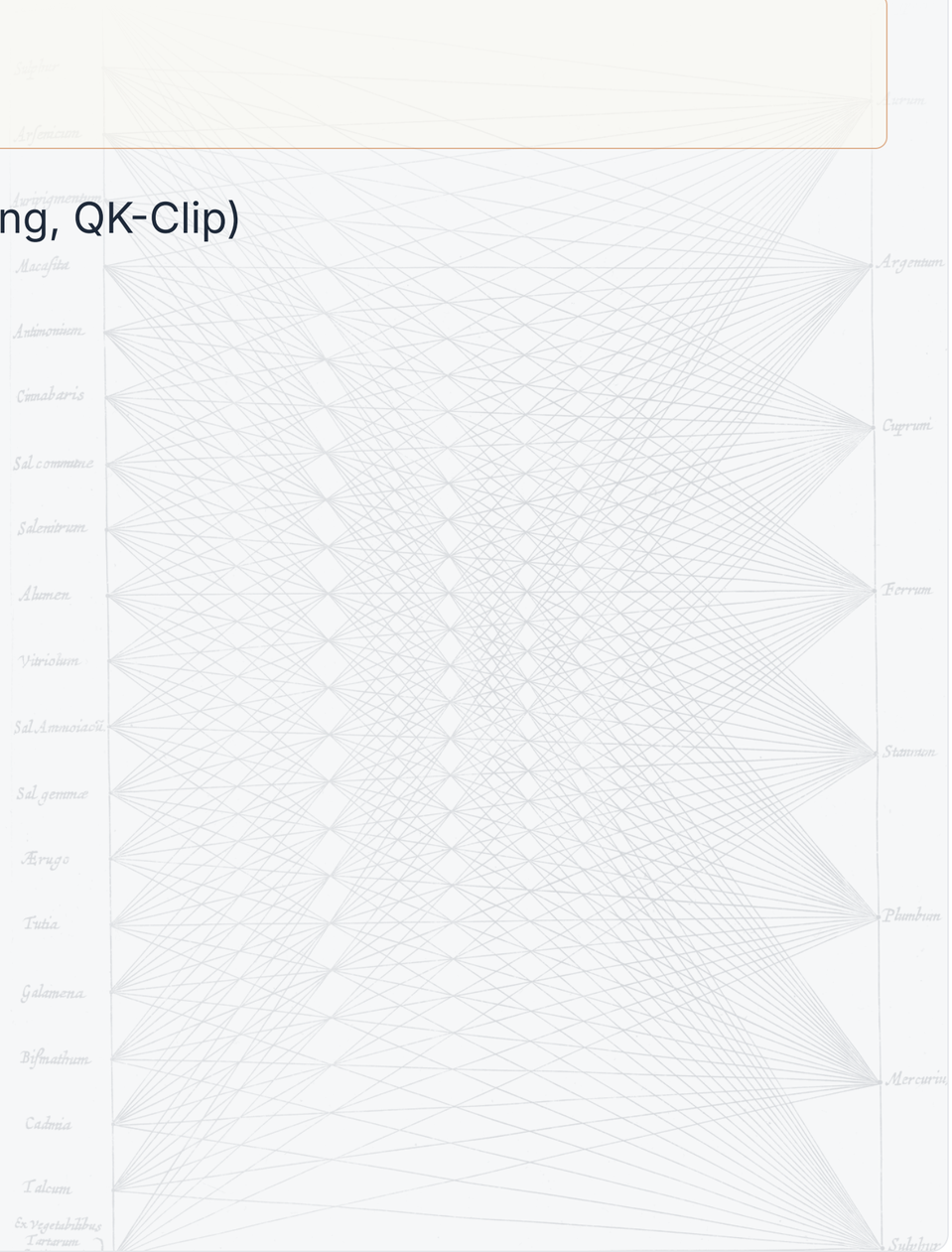
- DeepSeek V3: FP8 matrix multiplications, forward and backward (DeepSeek-AI, 2024b)
- Scales per 1×128 activation tile and 128×128 weight block
- Partial sums promoted to fp32 every 128 elements
- Embedding, head, norms and attention stay in bf16 or fp32
- MAI-Base-2, A.X K2: microscaling FP8 (MXFP8), 32-value blocks (Microsoft AI, 2026) (Baek et al., 2026)
- Local scales absorb outliers



The opening claim, decoded

K2: 15.5T tokens, zero loss spike

- Optimizer: MuonClip (Muon, weight decay, RMS matching, QK-Clip)
- Schedule: WSD, 10T tokens stable, 5.5T decay
- Monitoring: per-head max logit, every step
- Control: QK-Clip at $\tau = 100$
- Precision: bf16 weights, fp32 gradient accumulation



Six ways to keep a long run stable

1. AdamW: the tuned baseline to beat
2. Muon: shape-scaled updates, weight decay, bounded logits
3. Schedules: warmup, WSD decay legs, a batch ramp
4. Monitoring: pre-clip norm, max logit, spike score
5. Controls: each at its own tensor path
6. Precision: bf16 compute, fp32 master weights, local FP8 scales

To-dos until next week

Muddiest point

Do it right now! (2 min)

Before the tutorial

- Write down one question to discuss
- Think about the reflection questions

Self-study

- Read the reading slides and notes
- Read Bonus 3 before the Week 6 tutorial

Deadlines

- Quiz 2: Friday 13 November
- Fourth exercise: Wednesday 18 November, 22:00

Reflection questions

1. You are choosing between AdamW and a Muon+AdamW hybrid for a bf16 pretraining run with fp32 optimizer state. For each eligible matrix parameter, how many optimizer-state bytes does each choice require, and which other memory categories must remain unchanged in your comparison?
2. A WSD run reaches the end of its stable phase, and you need checkpoints adapted to two different token budgets without repeating that phase. Which checkpoints and schedule states would you save, where would you branch the decay legs, and what comparison would reveal the trade-off against separate cosine runs?
3. Two models show growing attention logits: one can modify its attention computation, while the other can intervene only through optimizer-side weight updates or the training loss. For each model, where would you place a stability control, what signal would it act on, and what limitation would you record?
4. A pretraining run produces gradients near `1e-8` and occasional large activations. Compare bf16 and fp16 for this run: which range or precision property drives your format choice, when would dynamic loss scaling be required, and which operations or states would you keep in fp32?
5. Run A shows an isolated loss spike with no preceding rise in pre-clip gradient norm. Run B shows pre-clip gradient norm rising for hours while loss remains smooth. What evidence would you inspect next for each run, and which interventions would you test without claiming the trace proves a cause?
6. Your small-vocabulary course model uses AdamW with `beta2=0.999`, and its second-moment estimate reacts slowly to rare large gradients. What change to `beta2` would you test, what response do you predict, and which logged comparison would distinguish faster adaptation from added noise?